System A — Event-Driven / Reactive Trading Agent (CS2 · BUFF)

> **Owner:** [Builder 1] **Scope:** React to game updates, news, and manipulation events faster and/or smarter than the rest of the market. Alpha source = *information*, not slow factor accumulation. **Sibling system:** System B (Positional Value/Trend). The two share a data layer and a normalization schema but run independently.

---

0. One-paragraph thesis

When Valve ships an update (e.g. the knife trade-up change that gutted knife prices and sent red-tier items like the MP9 | Asiimov and MP9 | Starlight Protector vertical), or when a hype/manipulation event forms, prices reprice violently over hours. The edge is being *early and correct* on which items a given event affects. **You will not win a pure speed race against entrenched, well-capitalized bots** that buy out an entire tier the instant an update drops. So this system is designed to win on *interpretation and breadth of monitoring* (catching the signal, mapping it to the right items, and acting within seconds–minutes), not on shaving milliseconds.

---

1. Market/platform assumptions

- **Primary venue:** BUFF163 (NetEase). Fees ≈ 2.5%, pricing in CNY, requires a Chinese account. This is the execution venue.
- **Trade cooldown:** every purchased item is locked for **~7 days (T+7)** before it can be re-listed/sold. This is a *hard* constraint, not a preference — see §5.
- **You can only go long.** No shorting. Every trade is a directional bet that resolves no sooner than 7 days out.
- **Books are thin.** Each item is its own micro-market. Size is limited by that item's real depth and daily volume.

---

2. Data layer (shared with System B)

2.1 The critical distinction: data API ≠ execution API

Every provider below returns *prices/volume/history*. **None of them place orders on BUFF for you.** Automated buying/selling is a separate problem handled in §6. Do not assume "we have the API" means "we can trade" — it means "we can see."

2.2 Provider recommendation (ranked for a BUFF-primary strategy)

1. **cs2.sh — SELECTED primary live feed.** It natively collects BUFF *listing prices, buy orders, and volume every few minutes*, plus float/fade ranges per item and per variant, builds OHLC at 5m/30m/1h/1d, and keeps a 3+ year archive. This is the only tier of data that lets you compute the volume/depth signals both systems need — not just a single median price.
2. **csmarketapi (the one you flagged) — good, especially for history.** Advertises 11+ years of sales history, 10+ markets, item metadata, and no Steam rate limits. Strong for deep backtests and cross-market context. **Action item before committing:** confirm its BUFF endpoints expose *per-item trade volume + listing counts + highest buy order*, not just lowest sell price. If it only gives price, it's a history/backup source, not your live signal source.
3. **pricempire** — ~5 years history, 56 markets, mature tooling. Good backtest/backup.
4. **cspriceapi / steamwebapi** — normalized multi-market JSON (BUFF, YouPin, Skinport, CSFloat…), buy orders, per-phase Doppler pricing. Good redundancy and cross-market divergence signals.

Recommended setup: `cs2.sh` as the live BUFF feed (native volume/depth/float) + `csmarketapi` *or* `pricempire` for the long historical series used in backtesting. Put a thin normalization layer in front so no single vendor is load-bearing — see §2.3.

2.3 Normalization schema (write this once, both systems import it)

Every source maps to a common record:

```
Item {
  market_hash_name        // canonical, matches Steam naming exactly
  variant                 // wear / Doppler phase / fade %  (nullable)
  buff_lowest_sell_cny
  buff_highest_buy_cny
  buff_listing_count      // sell-side depth proxy
  buff_buy_order_count    // bid-side depth proxy
  buff_volume_24h         // executed trades, NOT listings
  float_range             // if available
  ts                      // snapshot time (UTC)
  cross_market: { steam_usd, youpin_cny, skinport_usd, ... }
}
```

Persist snapshots to a time-series store (Timescale/Influx/Postgres). You need history for backtests and for the baselines the detectors in §4 subtract against.

---

3. What System A actually monitors

Two input streams feed the reactive engine:

- **Market stream** (from §2): abnormal price/volume/depth moves that front-run or confirm an event.
- **News/social stream** (from §7, the monitor agent): update leaks, official announcements, and China-side hype chatter.

The system fires when the two corroborate, or when either alone crosses a high-confidence threshold.

---

4. Event detection logic

> **Grounding:** System A trades **right-side / momentum** (Shared §6.1) — buy only on a *confirmed* move that can complete within one CD. The **Shared Market-Fundamentals & Indicator Library** supplies the confirmation filters and sizing discipline referenced below.

4.1 Update/patch reaction (the core play)

Trigger sequence:

1. **Signal in:** monitor agent surfaces a confirmed or high-probability update (see §7 for confidence scoring).
2. **Item mapping:** map the update to affected item sets. This is the hard, valuable part — maintain a **rules table** encoding known cause→effect relationships, e.g.: - "**weapon balance change (buff/nerf)**" → **the nerfed weapon's skins fall; its substitute weapon's skins rise** (and vice versa). This is empirically the cleanest, biggest mover: Nikolaenko's structural breaks land exactly on M4A1-S buff/nerf dates, with usage and price shifting between the M4A4/M4A1-S substitute pair (`/research/RESEARCH_INDEX.md`). Maintain an explicit **substitute-pair map** (M4A4↔M4A1-S, and other role-substitutes) so a balance leak instantly yields a buy list (the substitute) and a sell/avoid list (the nerfed gun). - "trade-up / contract pool change" → the tiers *fed into* the contract drop; the *output* tier and adjacent red-tier items spike. - "new case / collection" → older discontinued-collection items in the same weapon class tend to rise (scarcity narrative). - "operation / Armory pass re-releasing a map collection" → that collection's supply expectation rises → its items soften (this is your *sell/avoid* signal, and it's exactly the Cache/Hellhound supply logic System B trades).

Event-class discipline (empirically grounded — Shared §11): weight the rules table toward **balance/meta and supply-structural updates**, which cause *real, durable price breaks*. **Calendar/esports events (Steam sales, Majors, tournaments, operations) move *volume*, not daily price** (Pettersson) — so treat them as *liquidity/timing* signals (when books get thick, when to expect sale-window dips), **not** as price-entry triggers. Don't build reactive price bets on a Major starting; do note that a case release spikes volume ~34%. 3. **Liquidity check:** for each candidate item, pull current `listing_count` , `buy_order_count` , `volume_24h` . Skip anything you can't both enter *and later exit* at reasonable size (see §5.2). Apply the selection-grade liquidity floor (≥ ~10 trades/day; ≥3 valid buy orders — Shared §4.3) so you're not buying into an item you can't offload after CD. 4. **Confirm the move (right-side filter, Shared §3.1/§3.3):** prefer entries where the reaction shows **volume ↑ + price ↑ together** (healthy accumulation, pattern 3) and **band-width widening** (trend accelerating). **Avoid** price ↑ + volume ↓ (weak rally / one-wave pump — pattern 4) even on a real update; that's an exhausted move you'd be exit liquidity for. 5. **Act:** place buy orders on the ranked candidates within the size caps (momentum-chase ≤ 2 layers / 20% per item — Shared §5).

Because you can't out-speed the top bots, weight the rules table toward *second-order* items the fast bots miss — the adjacent/secondary items that reprice a few hours later once the obvious tier is bought out.

4.1a Market-data break detector (a second, non-social update detector)

Run a live **structural-break detector** (Bai–Perron / CUSUM) on the price/return feed. Nikolaenko showed these breaks land

precisely on balance-update dates, so this is a **market-data-based update alarm that doesn't depend on catching a social leak** — if the monitor misses a leak, an abnormal break still fires the reactive pipeline (and feeds the retrain trigger in Shared §12). Use it as corroboration with the social monitor, and as a backstop when leaks don't surface.

4.2 Manipulation/hype detection (as a RISK signal, not an entry — see §8)

Detect the *shape* of a forming pump so the system can **stay out or exit**, not chase it:

- Parabolic price with collapsing listing count and a fresh spike in social mentions = late-stage pump. **Block list**, do not buy.
- The EG-sticker pattern (≈100 RMB → 3,000+ RMB → ~400 RMB in weeks) is the canonical training example for the classifier's "danger" label.

---

5. Handling the 7-day cooldown (first-class constraint)

Every buy is illiquid for 7+ days. Encode this directly into decision-making:

5.1 Expected value must clear the lock

Only take a trade if:

```
E[price at t+7+] * (1 - buff_fee ≈ 0.025)  >  entry_price * (1 + required_edge)
```

`required_edge` should be padded because you are *forced* to hold through 7 days of event-decay risk. A patch spike often *fades* before your lock clears — so favor events where the repricing is expected to be **durable** (structural supply/scarcity changes) over transient hype pops that may be gone by day 7.

5.2 Position sizing vs. exit liquidity

Never buy more of an item than you can offload after the lock without crashing it. Rule of thumb:

```
max_units(item) = min(
   capital_cap_per_item / entry_price,
   k * buff_volume_24h            // k ≈ 0.2–0.5; you must be able to exit over a few days
)
```

Thin books are the trap: entering is easy, exiting a week later into a fading spike is where the losses live.

5.3 Cooldown-aware ledger

Track every lot with `unlock_time = buy_time + 7d`. The scheduler cannot even *consider* selling a lot before its unlock, and the risk engine must treat locked inventory as **non-liquid** capital when computing exposure.

---

6. Execution / automation layer (the hard part)

Your goal is **fully automated, no human in the loop**. Honest breakdown of what that requires on BUFF, because the data API does *not* provide it:

- **Option A — Official BUFF/NetEase API.** Sanctioned path. Costs ~$150/month and is gated to Chinese accounts. If you can operate a legitimate Chinese account, this is by far the most stable route to automated order placement and the one least likely to get nuked.
- **Option B — Unofficial session automation.** Authenticated session cookies + BUFF's internal buy/sell endpoints (open-source `buff163` buyer projects exist as references). This *works* and is how many of the bot accounts you've seen operate, **but**: it violates BUFF's ToS, BUFF actively invalidates sessions and blocks IPs, you'll fight Cloudflare/anti-bot and rate limits, and account/inventory/wallet bans are a live risk. If you go this way, isolate capital, expect to lose accounts, and build session-refresh + backoff from day one.

Recommendation: pursue Option A if a Chinese account is feasible; otherwise treat Option B as high-risk infrastructure with strict capital isolation. Either way, build the execution layer behind an interface (`place_buy`, `place_sell`, `get_inventory`, `get_wallet`) so the strategy code is decoupled from the (fragile) execution backend.

- **Idempotency & reconciliation:** every order gets a client-side ID; reconcile fills against the ledger every cycle. Reactive systems double-buy under latency if you skip this.
- **Kill switch:** a single flag that halts all buying (see §8.4).

---

7. Social-media monitor agent — SHARED infrastructure (built here,

consumed by both systems)

> **Does System A need this? Yes — it *is* System A's alpha source**, so the full build lives here. But the monitor is **shared infrastructure, not a System-A-private component** — market trends that System B trades are substantially driven by the same social narrative, so B is a first-class consumer too (see System B §7). Build it once, own it jointly, and have it publish a **tiered signal bus** (§7.5) that each system reads its own slice of. The *signals* are shared; the *decision logic* stays separate per system.

7.1 Sources

- **X/Twitter:** CS2 update leakers, dataminers, `@CounterStrike` official, major skin-trading accounts. Primary for update leaks and official announcements.
- **Chinese platforms:** Xiaohongshu/RedNote, Weibo, BUFF community threads, relevant Bilibili/Douyin posters. Primary for early hype/manipulation chatter (the EG-sticker crowd forms here first).
- **Official:** CS2 blog / release notes, Steam news, depot/build changes (dataminer feeds often surface these before the blog post).

7.2 Ingestion

- Prefer official/paid APIs where they exist (X API tier). For platforms that block automated access, you'll face the same anti-bot problems as BUFF; budget for it and respect that this is scraping gray-area. Rotate, backoff, cache.
- Poll on a tight cadence for high-signal accounts; slower for the long tail.

7.3 Processing pipeline

1. **Filter** to CS2-relevant posts (keyword + account allowlist).
2. **Classify** each item with an LLM into: `{update_leak, official_announcement, hype/manipulation, noise}` with a confidence score. Translate CN→EN in-pipeline.
3. **Entity-extract** the affected items/collections/weapons and the *direction* of expected impact.
4. **Corroborate** across sources — a leak echoed by 3 reputable accounts scores higher than a single unknown poster. This confidence score is what §4.1 step 1 and §5.1 consume.
5. **Emit** onto the shared bus (§7.5).

7.4 Guardrails

- Social content is **untrusted input.** Never let a scraped post directly trigger a trade — it feeds the *scoring* function only; the trade still requires the market-data corroboration and liquidity checks in §4–§5. This also protects you from planted/fake "leaks" designed to bait bots.
- Deduplicate reposts; decay old signals.

7.5 The shared tiered signal bus

The monitor publishes structured events at three confidence tiers; each system subscribes to the slice it needs.

- **Tier 1 — raw leaks / rumors** (fast, low-confidence). **System A only.** This is the reactive edge; System B ignores it.
- **Tier 2 — confirmed events** (official announcements, shipped updates, Armory re-releases). **Both systems.** A trades them; B uses them as a risk overlay (pause/exit on thesis break).
- **Tier 3 — attention / sentiment metrics** (per-item mention volume + sentiment, trended over time). **Primarily System B**, as a *leading entry feature* — rising attention on a still-flat-priced item precedes accumulation. A can read it to gauge crowding.

Event schema on the bus:

```
Signal {
  tier                    // 1 | 2 | 3
  type                    // update_leak | official_announcement | confirmed_update | hype | attention
  items[]                 // affected market_hash_names / collections
  direction               // bullish | bearish | unclear
  confidence              // 0..1 (corroboration-weighted)
  attention_score         // Tier 3: normalized mention volume vs. baseline
  sentiment               // Tier 3: -1..1
  first_seen_ts
  sources[]
}
```

Coupling caveats (apply to both systems): - *Shared failure mode:* if the bus stalls or emits garbage, both systems are affected. Each must degrade gracefully — B falls back to structural factors; A pauses reactive trading rather than acting on

stale signals. - *Correlation risk:* if A and B both pile into the same items off the same Tier-2 signal, you lose the diversification of running two strategies and double your exposure to one event. Give both systems read-visibility into each other's open positions so the builders don't unknowingly double up.

---

8. Risk management

8.1 Per-item limits

- Hard cap on capital per item (see §5.2).
- Volume-relative size cap so you can always exit.
- **Momentum-chase sizing ≤ 2 layers (20%) per item** (Shared §5) — right-side entries are the riskiest; never push to high deployment on a chase.
- Maintain a **block list** of items flagged as late-stage pumps (§4.2) — never buy these.

8.1a Regime awareness

- Read the shared `market_regime` signal (Shared §2). In a **bear regime, suppress reactive buys unless the event is durably structural** — chasing momentum into a falling market is catching a falling knife. Bull/sideways regimes get normal sizing.

8.2 Portfolio limits

- Max total exposure, and a **max locked-capital %** (since T+7 freezes capital, cap how much of the book can be simultaneously locked).
- Concentration limits per collection/weapon class (an update can hit a whole class at once).

8.3 Event-decay risk

- For each open lot, model expected value *at unlock*, not at entry. If the thesis was a transient spike, expect mean-reversion by day 7.
- Prefer durable/structural events over hype pops.

8.4 Operational risk

- **Kill switch** halting all buys instantly.
- Account-ban contingency: capital isolation across accounts; nothing catastrophic if one account dies.
- Vendor-failure handling: if the data feed stalls or diverges wildly across sources, pause trading rather than act on bad data.
- Reconciliation every cycle to catch double-fills and phantom orders.

8.5 Drawdown control

- Daily/weekly loss limits that trip the kill switch.
- Since realized P&L can only be measured after locks clear, track *marked* (mark-to-market on locked inventory) and *realized* separately.

---

9. Backtest & mock plan

- **Historical backtest** on multi-year data (csmarketapi/pricempire). Must model: BUFF fee, T+7 lock, realistic fills (you cannot fill more than the book/volume supports — cap fills at a fraction of depth), and slippage. A backtest that fills at the listed price and ignores the lock will look amazing and lose money live.
- **Event-study validation:** the reactive edge lives in rare events, so backtest specifically over historical update dates. Assemble a labeled set of past updates → item reactions and measure whether the rules table + monitor would have caught and profited from them *net of fees and lock*.
- **Forward paper trading:** run the live pipeline in shadow mode, logging simulated buys/sells at real observed prices, before touching real money.
- **Go-live gate:** only deploy real capital after the paper phase clears a pre-agreed edge threshold net of all costs.

---

10. Architecture summary

```
[X/CN scrapers] → Monitor Agent (§7) ─┐
                                      ├→ Reactive Engine (§4) → Risk Gate (§8) → Execution (§6) → BUFF
[cs2.sh / csmarketapi feed] → Normalizer (§2.3) ─┘         │
                                                           └→ Ledger (T+7 aware, §5.3)
```

Appendix P — Parked: "ride the early pump" (NOT for initial build)

Kept per request for future consideration; **excluded from the active strategy.**

The idea of buying a manipulated item early-and-cheap and dumping before the crash is *participating in* a pump-and-dump — your profit comes from the later retail buyers who take the loss (the same mechanism that trapped the EG-sticker loan-buyers). Beyond the ethics, it is the single riskiest play in the whole design: misjudge the top by days and the T+7 lock makes you the bagholder, in a legal environment tightening around exactly this behavior. If ever revisited, it should be gated behind strict, small capital isolation and treated as pure speculation, not strategy. For now, manipulation detection is used **only** as a *stay-out / exit* risk signal (§4.2, §8.1).